

Camunda 8 on ECS Fargate — Deployment Guide (eu-west-2)

As of 1 October 2026

Overview

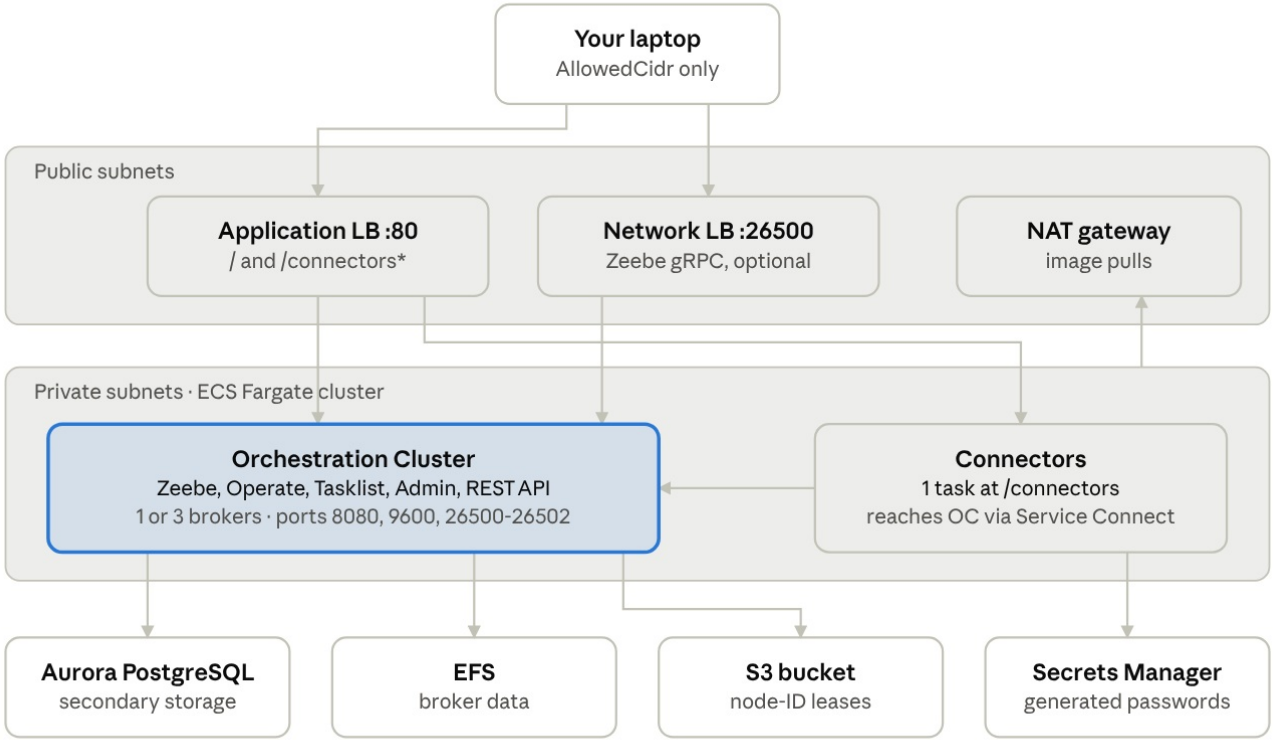
Two CloudFormation templates turn a blank AWS account in eu-west-2 into a self-service Camunda 8.9 product: you register it once, then anyone you allow can launch a full environment from Service Catalog in about 20 to 30 minutes.

File	What it is	Deployed by
camunda-servicecatalog-portfolio.yaml	Portfolio, product, launch role, launch constraint, user access, ECS service-linked role	You, once, as a normal CloudFormation stack
camunda-ecs-product.yaml	The Camunda environment itself	Service Catalog, each time someone launches the product

Each launch of the product creates, inside its own VPC:

- **Network:** a VPC across 3 AZs, public and private subnets, one NAT gateway, an S3 gateway endpoint.
- **Camunda:** an ECS Fargate cluster running the Orchestration Cluster (Zeebe, Operate, Tasklist, Admin, REST API) as 1 or 3 brokers, plus 1 Connectors task. The services find each other through ECS Service Connect.
- **Data:** Aurora PostgreSQL (secondary storage), EFS (broker data), an S3 bucket (broker node-ID leases), generated passwords in Secrets Manager.
- **Access:** an ALB on port 80 (/ for Camunda, /connectors* for Connectors) and an optional NLB for Zeebe gRPC on port 26500, both limited to one CIDR you choose.

Traffic enters only through the load balancers; Camunda runs in private subnets



Only the load balancers sit in public subnets; every Camunda task and data store is private, and outbound image pulls go through the NAT gateway.

The design follows Camunda's reference architecture ([camunda-deployment-references, stable/8.9, ecs-single-region-fargate](#)) with four simplifications: password database auth instead of IAM auth, no backup bucket, AWS-managed encryption keys instead of customer-managed ones, and a small Lambda that empties the S3 bucket so the product terminates cleanly.

Prerequisites

You need an admin-level IAM identity in the account, both template files, and enough Fargate vCPU quota in eu-west-2. Work in **Europe (London) eu-west-2** throughout: both templates refuse to deploy in any other region.

-] **Signed in with admin rights.** An IAM user or role with `AdministratorAccess` (or IAM Identity Center equivalent). Avoid the root user for day-to-day work.
-] **Region set to eu-west-2** in the console's region picker (top right).
-] **Template files downloaded:** `camunda-ecs-product.yaml` and `camunda-servicecatalog-portfolio.yaml`.
-] **Fargate vCPU quota checked.** Service Quotas → AWS Fargate → *Fargate On-Demand vCPU resource count*. A 3-broker launch at the default size needs 14 vCPU ($3 \times 4 + 2$). New accounts sometimes start lower. If yours is below 14, request an increase or launch with 1 broker first (6 vCPU).
-] **Your public IP known.** The load balancers only accept traffic from the CIDR you give at launch. Find your IP (for example at checkip.amazonaws.com) and use it as `x.x.x.x/32`.
-] **ARN of the person who will launch the product.** For yourself, IAM → Users or Roles → your identity → copy the ARN. With IAM Identity Center, use the `AWSReservedSSO_...` role ARN from IAM → Roles.
-] **Optional: AWS CLI v2** configured with `--region eu-west-2`, if you prefer the CLI commands shown alongside the console steps.

Step 1: Upload the product template to S3

Service Catalog reads the product template from an S3 URL, so put `camunda-ecs-product.yaml` in a private bucket in eu-west-2 and copy its object URL.

1. Open **S3 → Create bucket**.
 - Region: **eu-west-2**.
 - Name: something unique, for example `<your-account-id>-sc-templates-euw2`.
 - Keep **Block all public access** on and the other defaults. Create it.
2. Open the bucket → **Upload** → add `camunda-ecs-product.yaml` → **Upload**. A folder such as `camunda/` is optional.
3. Click the uploaded object and copy its **Object URL**. It looks like `https://<bucket>.s3.eu-west-2.amazonaws.com/camunda/camunda-ecs-product.yaml`. You need it in Step 2.

The bucket stays private. Service Catalog copies the template when the product version is created, using your permissions at that moment.

CLI equivalent:

```
aws s3 mb s3://<your-account-id>-sc-templates-euw2 --region eu-west-2
aws s3 cp camunda-ecs-product.yaml s3://<your-account-id>-sc-templates-euw2/camunda/ --region eu-west-2
```

Step 2: Deploy the Service Catalog portfolio stack

Deploy `camunda-servicecatalog-portfolio.yaml` once as an ordinary CloudFormation stack; it registers the product and takes about 2 minutes.

1. Open **CloudFormation → Create stack → With new resources (standard)**.
2. **Upload a template file** → choose `camunda-servicecatalog-portfolio.yaml` → Next.
3. Stack name: `camunda-servicecatalog`. Fill in the parameters:

Parameter	Value
ProductTemplateUrl	The Object URL from Step 1
ProductVersionName	Keep <code>v1-camunda-8.9</code>
EndUserPrincipalArn	ARN of the IAM user or role that will launch the product (Prerequisites)
OwnerName	Your team name, shown in the catalog
CreateEcsServiceLinkedRole	true in a fresh account; false if <code>AWSServiceRoleForECS</code> already exists in IAM → Roles

4. Next → leave stack options at their defaults → Next.
5. On the review page, tick **I acknowledge that AWS CloudFormation might create IAM resources** → Submit.
6. Wait for **CREATE_COMPLETE**. The **Outputs** tab shows `PortfolioId`, `ProductId` and `LaunchRoleArn`.

What this stack creates:

- **Portfolio** "Workflow Platforms" containing the product **Camunda 8 on ECS Fargate**, version `v1-camunda-8.9`.
- **Launch role and launch constraint.** Service Catalog provisions the product with this role, so end users don't need

EC2, ECS or RDS permissions of their own.

- **Portfolio access** for the principal you entered.
- **ECS service-linked role**, kept even if you delete this stack, because other ECS workloads rely on it.

CLI equivalent:

```
aws cloudformation deploy \
  --region eu-west-2 \
  --stack-name camunda-servicecatalog \
  --template-file camunda-servicecatalog-portfolio.yaml \
  --capabilities CAPABILITY_IAM \
  --parameter-overrides \
    ProductTemplateUrl=https://<bucket>.s3.eu-west-2.amazonaws.com/camunda/camunda-ecs-product.yaml \
    EndUserPrincipalArn=arn:aws:iam::<account-id>:role/<your-role>
```

Step 3: Give the launching user end-user access

The person launching the product needs two things: portfolio access (Step 2 already granted it) and the AWS managed policy `AWSServiceCatalogEndUserFullAccess`.

- If you launch it yourself with an admin identity, skip this step: `AdministratorAccess` already covers it.
- For an IAM user or role: IAM → Users (or Roles) → the identity → **Add permissions** → **Attach policies directly** → `AWSServiceCatalogEndUserFullAccess` → Add.
- For IAM Identity Center: add `AWSServiceCatalogEndUserFullAccess` to the permission set, then use that permission set's `AWSReservedSSO_... role ARN` as `EndUserPrincipalArn`.
- More people later: Service Catalog → Portfolios → Workflow Platforms → **Access** tab → Grant access, or update `EndUserPrincipalArn` on the portfolio stack.

Step 4: Launch the product

Launch from Service Catalog with your IP as `AllowedCidr` and everything else at its default; only change `BrokerCount` to 1 if you want a cheaper first run or your Fargate quota is below 14 vCPU.

1. Open **Service Catalog** → **Products** (end-user view). Select **Camunda 8 on ECS Fargate** → **Launch product**.
2. Provisioned product name: for example `camunda-poc`. Version: `v1-camunda-8.9`.
3. Fill in the parameters:

Parameter	Default	Recommendation
<code>AllowedCidr</code>	none (required)	Your public IP as <code>x.x.x.x/32</code>
<code>LoadBalancerScheme</code>	<code>internet-facing</code>	Keep. Use <code>internal</code> only with VPN or Direct Connect into this VPC
<code>EnableGrpcLoadBalancer</code>	<code>true</code>	Keep if you deploy BPMN or run job workers from your laptop; <code>false</code> saves one NLB
<code>BrokerCount</code>	3	3 for a highly available cluster; 1 for a cheap PoC
<code>OrchestrationTaskSize</code>	medium (4 vCPU / 8 GB)	small (2 vCPU / 4 GB) is enough for light testing
<code>OrchestrationImage</code>	<code>camunda/camunda:8.9.16</code>	Keep, or an ECR URI in eu-west-2
<code>ConnectorsImage</code>	<code>camunda/connectors-bundle:8.9.7</code>	Keep; stay on the same 8.9 minor as the orchestration image
<code>AdminEmail</code>	<code>admin@example.com</code>	Your email
<code>DBEngineVersion</code>	17.9	Keep
<code>DBInstanceClass</code>	<code>db.t3.medium</code>	Keep
<code>VpcCidr</code>	<code>10.190.0.0/16</code>	Change only if it clashes with a network you will peer with

4. Skip tags and notifications → **Launch product**.
5. Watch the status. It moves from **Under change** to **Available** in roughly 20 to 30 minutes. Aurora and the first Camunda start-up take most of that time.

To follow progress in detail, open **CloudFormation** → **Stacks**. The product's stack is named `SC-<account-id>-pp-<id>`; its **Events** tab shows each resource as it is created.

Step 5: Verify and log in

Once the product shows **Available**, open `CamundaWebUrl` and log in as `admin` with the password stored in Secrets Manager.

1. Service Catalog → **Provisioned products** → `camunda-poc` → **Outputs**:

Output	Use
CamundaWebUrl	Operate, Tasklist, Admin and the REST API
ConnectorsHealthUrl	Connectors health check
GrpcAddress	Zeebe gRPC endpoint for Desktop Modeler, job workers and zbctl (only if EnableGrpcLoadBalancer is true)
AdminPasswordSecretArn	Secret holding the admin password
ConnectorsPasswordSecretArn	Secret holding the connectors user password
DatabaseEndpoint	Aurora writer endpoint
EcsClusterName, LogGroupName, VpcId	For troubleshooting

2. **Get the admin password:** Secrets Manager → open the secret named in AdminPasswordSecretArn → **Retrieve secret value**.
3. **Log in:** open CamundaWebUrl (plain http://, port 80) → user admin.
4. **Check health:**
 - ConnectorsHealthUrl returns {"status": "UP"}.
 - ECS → Clusters → the cluster in EcsClusterName → both services show all tasks **Running** and **Healthy**.
 - EC2 → Target Groups → each target is **healthy**.
 - CloudWatch → Log groups → LogGroupName: the brokers report joining the cluster and their partitions becoming healthy.
5. **Deploy something:** in Camunda Desktop Modeler, choose *Camunda 8 Self-Managed*, cluster endpoint = GrpcAddress (with http://), authentication *Basic*, user admin and its password. Deploy a simple BPMN and start an instance; it appears in Operate.

CLI to fetch the admin password:

```
aws secretsmanager get-secret-value --region eu-west-2 \
  --secret-id <AdminPasswordSecretArn> --query SecretString --output text
```

Operating it

Ship changes as new product versions, terminate environments when you're not using them, and delete the final Aurora snapshot afterwards.

Publish a new version (for example a newer Camunda 8.9 patch):

1. Edit camunda-ecs-product.yaml (for example the OrchestrationImage default) and upload it to S3 under a new key, such as camunda-ecs-product-v2.yaml.
2. Service Catalog → Administration → Products → **Camunda 8 on ECS Fargate** → **Versions** → **Create new version** → *Use a CloudFormation template file* via the new S3 URL → name v2-camunda-8.9.x.
3. Users move an existing environment with **Provisioned products** → **Actions** → **Update** and pick the new version. Mark the old version inactive once nobody needs it.

Change parameters on a running environment: Provisioned products → Actions → **Update** → same version, new values. Changing BrokerCount on a running cluster is not supported by Camunda's partition layout; launch a new environment instead.

Terminate: Provisioned products → Actions → **Terminate**. It removes everything except:

- **A final Aurora snapshot** (RDS → Snapshots → Manual). Delete it if you don't need the data.
- **Generated secrets** are scheduled for deletion with a 30-day recovery window, and aren't billed while pending deletion.

Cost while running, at the default settings:

Item	Size
Fargate	14 vCPU / 28 GB (3 brokers + Connectors); 6 vCPU / 12 GB with 1 broker
Aurora PostgreSQL	1 × db.t3.medium plus storage and I/O
NAT gateway	1, hourly plus per-GB
Load balancers	1 ALB, plus 1 NLB if gRPC is enabled
EFS, S3, Secrets Manager, CloudWatch Logs	Small at PoC volumes

Check current London prices in the AWS Pricing Calculator before leaving it running for long.

Troubleshooting

Most failures show up first in the product stack's **Events** tab (CloudFormation → SC-<account-id>-pp-<id>) and then in the ECS service's **Events** and stopped-task reasons.

Symptom	Likely cause	Fix
Portfolio stack fails on <code>EcsServiceLinkedRole</code>	The role already exists	Redeploy with <code>CreateEcsServiceLinkedRole = false</code>
Launch fails at once with a rule assertion error	Console region is not eu-west-2	Switch to Europe (London) and launch again
<code>DbCluster</code> fails: engine version not available	17.9 not offered in the account	Check RDS → Create database for the newest Aurora PostgreSQL 17.x and launch with that <code>DBEngineVersion</code>
ECS service fails with <i>You've reached the limit on the number of vCPUs</i>	Fargate vCPU quota too low	Request a quota increase, or launch with <code>BrokerCount = 1</code> and/or <code>OrchestrationTaskSize = small</code>
Tasks stop with <code>CannotPullContainerError</code>	Docker Hub rate limit or wrong image URI	Wait and retry, or push the images to ECR in eu-west-2 and use those URIs
Tasks stop with <code>ResourceInitializationError</code> mentioning EFS or secrets	Mount target or IAM not ready yet, or a typo'd ECR URI	Check the stopped task's reason; retry the launch if it was timing
Service fails with a circuit-breaker rollback	Camunda container keeps crashing	Read <code>LogGroupName</code> in CloudWatch for the first error (database connection, cluster size mismatch)
Brokers run but never become healthy	Brokers can't reach each other on 26502	Confirm all 3 tasks are in the same Service Connect namespace and <code>BrokerCount</code> matches the running count
Browser times out on <code>CamundaWebUrl</code>	Your IP changed or differs from <code>AllowedCidr</code>	Update the provisioned product with your current IP/32
Login page loads but <code>admin</code> is rejected	Wrong secret copied	Use the secret from <code>AdminPasswordSecretArn</code> , not the connectors or DB secret
Terminate fails on the S3 bucket	Bucket-cleaner Lambda failed	Empty the bucket in the S3 console, then terminate again

Use **ECS Exec** to look inside a running container: it is enabled on both services, and the task roles allow it.